

Proyecto 1: Risk R - Entregas 0 y 1

Departamento de Ingeniería de Sistemas

Estructuras de Datos

Nikolas Hannee Garcia Cure

Nicolas Moreno Alvarez

4 de septiembre de 2026

1. Introducción

El presente documento detalla el diseño, estructura e implementación de las Entregas 0 y 1 del proyecto de la asignatura Estructuras de Datos, el cual consiste en la sistematización del juego de mesa Risk R. La Entrega 0 se enfoca en el desarrollo de la interfaz de usuario por línea de comandos, asegurando la validación de instrucciones y parámetros requeridos. Por su parte, la Entrega 1 corresponde al Componente 1, centrado en la gestión y configuración del juego mediante estructuras de datos lineales. Esto abarca la inicialización del tablero con sus territorios, la distribución de los ejércitos iniciales y el desarrollo por turnos, incluyendo las fases de obtención de unidades, ataques y fortificación de posiciones.

2. Diseño de Tipos Abstractos de Datos (TAD)

Para mantener la modularidad, la reutilización de código y el principio de ocultación de la información, el sistema fue modelado a través de Tipos Abstractos de Datos (TAD). A continuación, se presenta el diagrama general de relación estructural y el diseño detallado de cada TAD siguiendo el esquema formal requerido para la primera entrega.

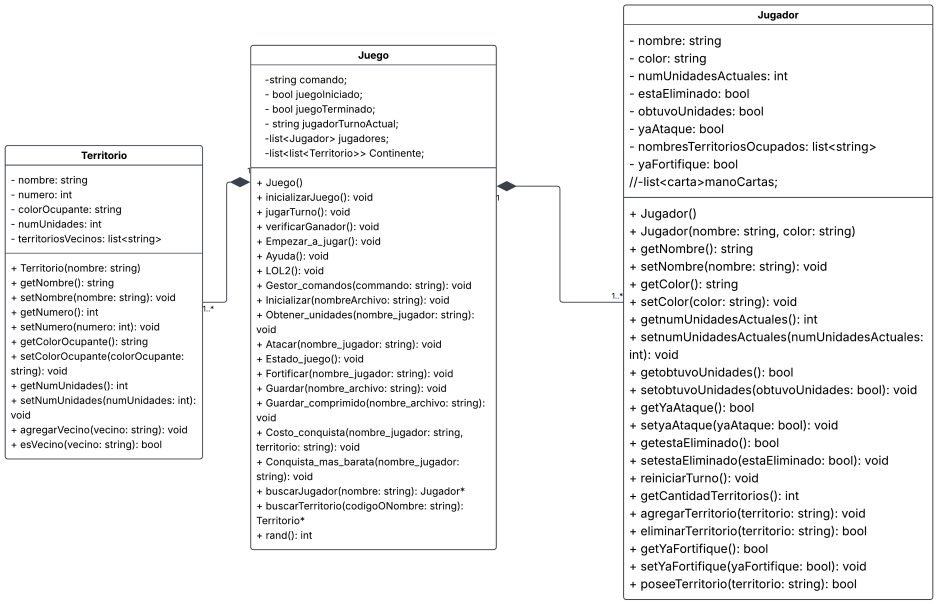


Figura 1: Diagrama de diseño y relación de los TAD del Componente 1.

2.1. TAD Territorio

Descripción: Abstrae el concepto de un territorio geográfico en el tablero de Risk, permitiendo gestionar su información intrínseca, el jugador quien lo ocupa y la cantidad de unidades del ejércitos dentro de él.

Conjunto mínimo de datos:

- `codigo`, entero, identificador numérico único del territorio (basado en los 42 territorios del tablero).
- `nombre`, cadena de caracteres, contiene el nombre del territorio.
- `continente`, cadena de caracteres, continente al que pertenece.
- `colorJugador`, cadena de caracteres, identificador del color del jugador que lo ocupa actualmente.
- `numUnidades`, entero, cantidad de unidades de infantería posicionadas.

Comportamiento operaciones:

- `crear(codigo, nombre, continente)`, inicializa un territorio vacío con cero unidades.
- `setJugador(color)`, cambia el color del jugador que ocupa el territorio por el recibido como parámetro.
- `adicionarUnidades(cantidad)`, suma la cantidad recibida a las unidades actuales.
- `reducirUnidades(cantidad)`, resta la cantidad recibida de las unidades actuales.
- `entero getNumUnidades()`, retorna la cantidad actual de unidades en el territorio.
- `cadena getColorJugador()`, retorna el color del jugador que controla el territorio.

Pontificia Universidad JAVERIANA Bogotá

2.2. TAD Jugador

Descripción: Abstrae el concepto de un jugador participante en la partida, gestionando su identidad, color distintivo y el estado de sus territorios conquistados y acciones en cada turno.

Conjunto mínimo de datos:

- `nombre`, cadena de caracteres, contiene el nombre del jugador (máximo 8 caracteres).
- `color`, cadena de caracteres, contiene el color asignado que distingue al jugador (p. ej. verde, azul, rojo).
- `territoriosOcupados`, lista de referencias o códigos de los territorios bajo el control del jugador.

Comportamiento operaciones:

- `crear(nombre, color)`, crea un jugador con los datos recibidos.
- `agregarTerritorio(codigo)`, añade un territorio a la lista de control al ser conquistado o inicializado.
- `removerTerritorio(codigo)`, elimina un territorio de la lista cuando se pierde tras un ataque enemigo.
- `entero contarTerritorios()`, retorna el total de territorios controlados para calcular bonificaciones.
- `cadena getNombre()`, retorna el nombre del jugador.
- `cadena getColor()`, retorna el color del jugador.

2.3. TAD Juego

Descripción: Abstrae el estado general y las mecánicas de la partida de Risk segun las reglas. Actúa como el controlador principal que gestiona la lista de jugadores y el tablero global.

Conjunto mínimo de datos:

- `jugadores`, lista de objetos Jugador, contiene a todos los participantes activos.

Pontificia Universidad JAVERIANA Bogotá

- `mapaTerritorios`, lista de objetos Territorio, contiene la totalidad de los 42 territorios.
- `juegoInicializado`, booleano, indica si la partida ya ha cargado los datos desde el archivo correctamente.

Comportamiento operaciones:

- `crear()`, inicializa el gestor con las estructuras lineales vacías.
- `inicializarJuego(archivo)`, lee el archivo de texto (`archivo_inicio.txt`) y configura la partida inicial.
- `obtenerUnidades(jugador)`, calcula y asigna los ejércitos de bonificación en el turno.
- `atacar(jugador)`, coordina la simulación de ataque, el lanzamiento de dados y los traspasos de unidades.
- `fortificar(jugador)`, permite el movimiento táctico de tropas al finalizar el turno.
- `mostrarEstado()`, imprime en consola el reporte general de ocupación del tablero.

3. Descripción de Entradas, Salidas y Condiciones

Esta sección detalla en las entradas, salidas esperadas (basadas en la consola de ejecución real) y condiciones de ejecución tanto para el procedimiento principal como para las operaciones auxiliares (comandos) requeridas en el componente 1.

3.1. Procedimiento Principal (main)

- **Entradas:** Recepción de comandos de texto introducidos por el usuario mediante teclado a través de la consola interactiva (ej. al recibir el prompt `$Comando:`). El sistema es tolerante a variaciones de mayúsculas y minúsculas.

- **Salidas:** Presentación del banner de bienvenida (`----------BIENVENIDO AL JUEGO DE RISK----------`), el indicador de línea de comando (`$Comando:`) y los mensajes de error por comandos inválidos (ej. `Comando no reconocido. Escriba 'ayuda' para ver la lista de comandos.`).
- **Condiciones:** El sistema debe mantenerse en un ciclo de escucha activo hasta que se ingrese explícitamente el comando de terminación.

3.2. Comando: ayuda

- **Entradas:** El comando `ayuda` (reconoce variaciones como `AyUda`).
- **Salidas:** Despliega la lista de todos los comandos disponibles en el sistema.

```
----------AYUDA----------
inicializar archivo_inicio.txt
obtener_unidades nombre_jugador
atacar nombre_jugador
fortificar nombre_jugador
estado_juego
guardar <nombre_archivo>
guardar_comprimido <nombre_archivo>
costo_conquista <nombre_jugador> <territorio>
conquista_mas_barata <nombre_jugador>
```

- **Condiciones:** Se puede ejecutar en cualquier momento, sin importar el estado del juego.

3.3. Comando: inicializar archivo_inicio.txt

- **Entradas:** El comando junto al nombre de un archivo de texto plano.
- **Salidas:** Si es exitoso, la salida exacta es: `(Comando correcto) El juego se ha inicializado correctamente.` Si falla, muestra el tipo de error (archivo vacío, erróneo o formato incorrecto).

- **Condiciones:** El juego no debe haber sido inicializado previamente.

3.4. Comando: obtener_unidades nombre_jugador

- **Entradas:** El comando con el identificador del jugador (ej. `obtener_unidades nikolas`). De forma interactiva se solicitan los códigos de los territorios (`Ingrese el codigo del territorio:`) y las cantidades a asignar (`Ingrese la cantidad de unidades para X:`).
- **Salidas:** Muestra un panel informativo (`========== OBTENCION DE UNIDADES ==========`) con:
  - Bonificaciones desglosadas (`Continente controlado: America del Norte (+5 unidades)`).
  - Resumen aritmético de unidades a reclamar.
  - Lista numerada de los territorios actuales del jugador y sus tropas.
  - Validaciones en tiempo real (ej. `La cantidad debe estar entre 1 y 11. Intente nuevamente.`).
  - Mensaje de culminación: `(Comando correcto) El jugador [nombre] ha terminado de reclamar y ubicar sus unidades.`
- **Condiciones:** Debe ser el turno del jugador y el juego debe estar inicializado (si se ingresa un jugador fuera de turno, se muestra: `(Jugador fuera de turno) No es el turno del jugador...`).

3.5. Comando: atacar nombre_jugador

- **Entradas:** El comando con el nombre del jugador. Interactivamente solicita `Territorio de origen` y `Territorio de destino`. Acepta `fin` en el origen para terminar la fase. Solicita confirmación de continuación (`Desea continuar atacando? (s/n):`) y cantidad de tropas a mover si hay conquista.
- **Salidas:** Panel `========== FASE DE ATAQUE ==========`. Despliega:

- Errores de validación capturados: `El territorio X no es vecino de Y.` o `No puede atacar un territorio propio.`
- Reporte de combate: `--- Combate: 1.1 (14 unidades) vs 5.6 (3 unidades) ---`.
- Resultados de dados (rojos y blancos) y pérdidas en la ronda.
- Mensaje de victoria: `El territorio X ha sido conquistado!.`
- Salida final: `(Comando correcto) El jugador [nombre] ha terminado de atacar.`

- **Condiciones:** El jugador debe poseer el turno actual, controlar el territorio de origen y tener más de una unidad en él.

3.6. Comando: fortificar nombre_jugador

- **Entradas:** Comando y nombre del jugador. Interactivamente solicita dos territorios vecinos controlados y la cantidad de tropas a desplazar.
- **Salidas:** Mensaje de éxito informando la reubicación táctica, o reportes de falla.
- **Condiciones:** El jugador debe haber ejecutado previamente la fase de ataque (`atacar`) y poseer territorios aptos para el movimiento táctico.

3.7. Comando: estado_juego

- **Entradas:** `estado_juego` (sin parámetros).
- **Salidas:** Banner `========== ESTADO DEL JUEGO ==========`. Muestra el número total de jugadores, estadísticas por jugador (nombre, color, territorios conquistados, unidades totales), el jugador que posee el `Turno actual`, y una lista tabulada agrupada por continentes detallando cada territorio con su código, color del ocupante y número de unidades.
- **Condiciones:** La partida debe estar en curso.

3.8. Comando: salir

- **Entradas:** Ninguna.
- **Salidas:** No presenta salidas por pantalla (termina la ejecución del ejecutable `.exe`).
- **Condiciones:** Ninguna.

4. Detalles Técnicos: Restricciones de Iteración y Control de Tipos

Para efectuar cualquier recorrido a lo largo de las estructuras de datos lineales (listas de jugadores, iteración sobre los 42 territorios o manipulación interna de colecciones), **se determinó como regla arquitectónica el uso ineludible de iteradores explícitos** (`std::list<T>::iterator`).

Esto significa de forma categórica que en ninguna porción de las funciones implementadas se hizo uso de sentencias `foreach`, y paralelamente, **se omitió por completo el uso de la palabra reservada `auto`**.

```
// Ejemplo ilustrativo de la convenci n de recorrido aplicada:
for (std::list<Territorio>::iterator it = listaTerritorios.begin(); it !=
    listaTerritorios.end(); ++it) {
    // Procesamiento con iterador expl cito
}
```

La rigidez adoptada en este modelo iterativo garantiza un control y seguimiento explícito sobre los punteros de la memoria e impide enmascaramientos de tipos de datos, fortaleciendo la calidad formativa en el manejo profundo de C++ y sus estructuras lineales.

5. Uso de Herramientas de IA Generativa

En cumplimiento de los lineamientos del proyecto referentes al uso de herramientas de inteligencia artificial generativa, a continuación se declaran los *prompts* principales utilizados para apoyar el desarrollo de estructuras lógicas y condicionales específicas dentro del código

fuente. Cabe destacar que el código provisto por la IA fue adaptado, revisado y acoplado de forma manual para asegurar el cumplimiento de las normativas de la asignatura.

- **Generación de la estructura de vecinos:**

  "ayudame a hacer la funcion de obtenerMapaVecinos donde le llega un map con una key que es un string y su dato es un vector de strings con este documento"

- **Estandarización de validaciones de estado del juego:**

  "quiero que añadas las conciciones para entrara ala clase que es

  ```
  if (!juegoIniciado) {
      cout << "(Juego no inicializado) Esta partida no ha sido inici
      correctamente." << endl;
      return;
  }
  if (juegoTerminado) {
      cout << "(Juego terminado) Esta partida ya tuvo un ganador." <
      return;
  }
  ```

  copia y pega esto en las clases que sea nesesaria"

- **Manejo de excepciones en la simulación de combates:**

  "ayudame a agregar try cach en la funcion de atacar donde sea nesesario"

- **Mapeo de identificadores y bonificaciones:**

  "ayudame a hecer la funcion nombreContinente que dependiendo de el numero retorne el pais correspondiente y otro que reciva un numero que es el territorio y retune el bonus dependiendo de cada 1"

- **Organización modular de encabezados:**

  “las libreias se ponen en el .cpp o el .h”

- **Organización del documento en APA:**

  “ ayudame a transformar este documento de word en uno de latex en formato apa ”

- **Control de condiciones del encabezado:**

  “ Asegurame que en ningun punto use for each, en vez de usar iteradores para recorrer las listas ”

Pontificia Universidad
JAVERIANA
Bogotá

6. Anexos: Pruebas de Ejecución

En esta sección se presentan las capturas de pantalla correspondientes a las pruebas de ejecución del programa, evidenciando las salidas reales en consola para cada uno de los comandos principales implementados.

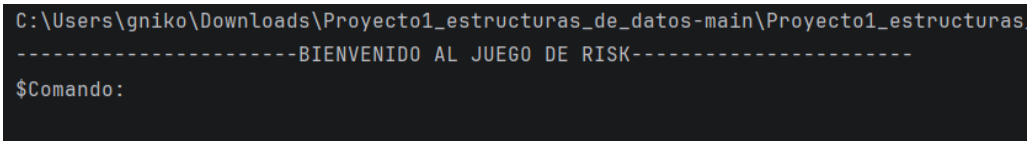


Figura 2: Prueba de ejecución: Bienvenida y menú de inicio.



Figura 3: Prueba de ejecución: Despliegue del comando `ayuda`.

```
$Comando:
inicializar archivo_inicio.txt

(Comando correcto) El juego se ha inicializado correctamente.
```

Figura 4: Prueba de ejecución: Validación del comando `inicializar`.

```
========== OBTENCION DE UNIDADES ==========
Jugador: nikolas

  Continente controlado: America del Norte (+5 unidades)
  Continente controlado: America del Sur (+2 unidades)

Resumen de unidades adicionales:
  - Por territorios (14 / 3): 4 unidades
  - Por continentes: 7 unidades
  - Total a reclamar: 11 unidades

Territorios del jugador:
  1. 1.1 - 3 unidades
  2. 1.2 - 3 unidades
  3. 1.3 - 3 unidades
  4. 1.4 - 3 unidades
  5. 1.5 - 3 unidades
  6. 1.6 - 3 unidades
  7. 1.7 - 3 unidades
  8. 1.8 - 2 unidades
  9. 1.9 - 2 unidades
  10. 2.1 - 2 unidades
  11. 2.2 - 2 unidades
  12. 2.3 - 2 unidades
  13. 2.4 - 2 unidades
  14. 3.1 - 2 unidades

Unidades restantes por asignar: 11
Ingrese el codigo del territorio:
```

Figura 5: Prueba de ejecución: Interfaz y validaciones del comando `obtener_unidades`.

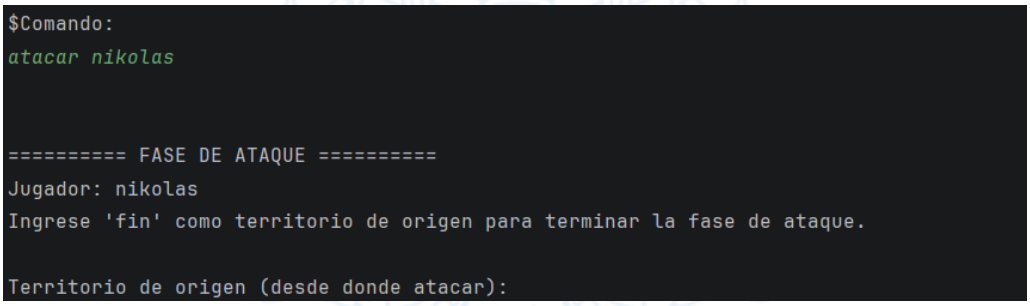


Figura 6: Prueba de ejecución: Flujo de simulación de combate del comando `atacar`.

Pontificia Universidad JAVERIANA Bogotá